

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech
Course Code & Course Name	CSA5115 – Cryptography and Network Security
Academic Year / Batch	2025-2029
Faculty Name	Dr. T. Sangeetha
Assignment Title	Evaluation and Design of an Integrated Public Key and Authentication Security System
Date of Issue	02-09-2026
Date of Submission	02-09-26
Maximum Marks	100
Course Outcome(s) – CO	CO2: Analyse Public Key crypto system strategies using RSA and Key Exchange for Encryption algorithm to strengthen information security. CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.
Bloom's Taxonomy Level	BL5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 8 – Decent Work and Economic Growth; SDG 12 – Responsible Consumption and Production
Industry / Societal Relevance	Directly relevant to the financial services industry — secure key management, authenticated transactions and reliable digital communication are core requirements for any organization handling sensitive customer and financial data.

B. Assignment Problem / Challenge

This assignment requires the evaluation and comparison of public-key cryptographic and authentication security mechanisms for a real-world financial-sector case, involving the application of course concepts (rather than reproduction of theory), analysis of requirements and constraints, comparison of alternative solutions, use of modern cryptographic tools, interpretation of results, and a justified engineering decision.

C. Problem Statement

Problem / Case: Evaluation and Design of an Integrated Public Key and Authentication Security System

A financial services organization operates a distributed digital environment connecting 500+ employees, branch offices, remote users, transaction processing systems, database servers and internal application servers. The organization exchanges sensitive customer and financial information across potentially untrusted communication networks. A recent security assessment has identified concerns related to weak key management, insecure key exchange, unauthorized access, modification of transaction data, identity impersonation and lack of reliable verification of electronic transactions. The organization requires secure cryptographic mechanisms to strengthen information security, trusted communication and reliable digital transactions.

The organization is considering three cryptographic security approaches:

- Strategy A – Conventional Public Key Security: RSA Encryption + Public/Private Key Pair.
- Strategy B – Secure Key Establishment: RSA Encryption + Diffie–Hellman Key Exchange + Hashing.
- Strategy C – Trusted Cryptographic Security: RSA + Diffie–Hellman Key Exchange + Cryptographic Hash + Digital Signature + X.509 Authentication.

As a Cryptography and Network Security Analyst, the task is to examine and implement the three proposed strategies by demonstrating suitable key generation, public/private key creation, key exchange, encryption, decryption, hashing and digital signature generation and verification; to

examine how the selected mechanisms protect confidential transaction information and establish authenticity and integrity; to analyse the role of RSA and key-exchange mechanisms in secure communication; to demonstrate attack scenarios such as unauthorized key acquisition, message modification and impersonation and evaluate resistance to these threats; and finally to evaluate Strategies A, B and C on confidentiality, integrity, authentication, non-repudiation, key-management complexity, computational efficiency, scalability, security strength, resistance to attacks and reliability, and recommend the most appropriate approach.

D. Requirements and Constraints

- Functional requirements: Confidentiality, integrity, authenticity and non-repudiation of financial transaction data.
- Performance requirements: Acceptable key-exchange, encryption/decryption and signature-verification latency for 500+ concurrent users.
- Technical constraints: Compatibility with existing transaction-processing, database and application servers; support for RSA, Diffie–Hellman and X.509 (PKIX) standards.
- Cost, Time: Licensing, deployment and administration cost of PKI infrastructure within a limited rollout period.
- Resources: Availability of trained security administrators to manage keys and certificates.
- Reliability: High availability of authentication and transaction-verification services.
- Sustainability & Ethical considerations: Responsible handling of customer and financial data (SDG 12), supporting decent, trustworthy digital work (SDG 8).
- Standards/codes: Compliance with RFC 5280 (X.509 PKIX), RFC 3526/2631 (Diffie–Hellman), PKCS#1 (RSA) and NIST FIPS 180-4 (SHA-256).
- Security/privacy: Protection of private keys, prevention of man-in-the-middle and replay attacks, and safeguarding of digital identity.

E. Student Work

1. Problem Understanding and Formulation

What is the problem? The financial organization must protect key management, key exchange, transaction confidentiality, integrity and authentication against unauthorized access, tampering and impersonation, while choosing among three candidate cryptographic strategies of increasing sophistication (A, B and C).

Expected outcomes: A working implementation and comparison of the three strategies, evaluated on confidentiality, integrity, authentication, non-repudiation, key-management complexity, computational efficiency, scalability, security strength, resistance to attacks and reliability, culminating in a justified recommendation.

Information/data available: The organizational profile (500+ users, branch offices, transaction and database servers), the three candidate strategies and their constituent mechanisms, and standard published properties of RSA, Diffie–Hellman, SHA-256 hashing, digital signatures and X.509 certificates.

Assumptions made: A moderate-scale distributed financial network; RSA-2048 used for encryption and digital signatures; 2048-bit MODP group used for Diffie–Hellman; SHA-256 used as the hashing standard; simulated/representative transaction data used in the absence of live production data.

Constraints to be satisfied: The chosen solution must balance security strength against cost, computational overhead and key-management complexity, while remaining compliant with relevant cryptographic standards.

2. Application of Course Knowledge

- The following course concepts (CO2, CO3) are applied to formulate and evaluate the three strategies:
- RSA public-key cryptosystem: asymmetric encryption using a public/private key pair, where data encrypted with the public key can only be decrypted with the corresponding private key, providing confidentiality.

- Diffie–Hellman key exchange: enables two parties to establish a shared secret key over an insecure channel without ever transmitting the key itself, addressing the key-distribution weakness of RSA-only schemes.
- Cryptographic hash algorithms (SHA-256): generate a fixed-length message digest that verifies data integrity; any alteration of the message produces a completely different digest.
- Digital signature schemes (RSA-based): the sender signs the hash of a message with its private key; the receiver verifies it with the sender's public key, providing authentication, integrity and non-repudiation.
- X.509 authentication framework: a public-key certificate standard binding an entity's identity to its public key through a Certificate Authority (CA), enabling scalable, trusted authentication across the organization.

Illustrative reasoning – key establishment: RSA-only key transport (Strategy A) requires the sender to already possess the receiver's authentic public key and offers no forward secrecy, whereas Diffie–Hellman (Strategy B and C) allows a fresh shared session key to be negotiated for every session, which is a substantially stronger guarantee against long-term key compromise.

3. Solution / Design / Methodology

Three implementations were built corresponding to Strategies A, B and C, each layering additional cryptographic mechanisms onto the previous one, using Python's cryptography library.

Strategy A – Conventional Public Key Security

RSA-2048 key pairs are generated for sender and receiver. The sender encrypts a transaction message using the receiver's RSA public key (OAEP padding); the receiver decrypts it with the corresponding private key.

```
"""
Strategy A - Conventional Public Key Security
RSA Encryption + Public/Private Key Pair
"""
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# 1. Key generation
receiver_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
receiver_public = receiver_key.public_key()
```

```

# 2. Encryption with receiver's public key
message = b"Fund transfer: A/C 100234 -> A/C 100987, Amount INR 250000"
ciphertext = receiver_public.encrypt(
    message,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
                  algorithm=hashes.SHA256(), label=None),
)
print("Ciphertext (hex):", ciphertext.hex()[:60], "...")

# 3. Decryption with receiver's private key
plaintext = receiver_key.decrypt(
    ciphertext,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
                  algorithm=hashes.SHA256(), label=None),
)
print("Decrypted OK:", plaintext == message)

# 4. Attack scenario - unauthorized key acquisition
attacker_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
try:
    attacker_key.decrypt(ciphertext, padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(), label=None))
except Exception:
    print("Attacker without the correct private key cannot decrypt (as expected)")

```

Strategy B – Secure Key Establishment

Diffie–Hellman key exchange establishes a shared secret between sender and receiver; SHA-256 derives a symmetric session key from it, which is used with RSA-signed hashing to detect message modification. RSA is retained for authenticated transport of the session parameters.

```

"""
Strategy B - Secure Key Establishment
RSA + Diffie-Hellman Key Exchange + Hashing
"""
import hashlib
from cryptography.hazmat.primitives.asymmetric import dh, rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

# 1. Shared DH parameters (2048-bit) and key exchange
parameters = dh.generate_parameters(generator=2, key_size=2048)
sender_priv = parameters.generate_private_key()
receiver_priv = parameters.generate_private_key()
sender_shared = sender_priv.exchange(receiver_priv.public_key())
receiver_shared = receiver_priv.exchange(sender_priv.public_key())
print("Shared secrets match:", sender_shared == receiver_shared)

# 2. Derive a symmetric session key via HKDF (SHA-256)
session_key = HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
                    info=b"txn-session").derive(sender_shared)

# 3. RSA-signed hash for integrity of the DH public value (prevents MITM)
rsa_priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
dh_pub_bytes = sender_priv.public_key().public_numbers().y.to_bytes(256, 'big')
digest = hashlib.sha256(dh_pub_bytes).digest()

```

```

signature = rsa_priv.sign(digest, padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256())
print("DH public value hash signed for authenticity:", signature.hex()[:40], "...")

# 4. Attack scenario - message modification detection
original = b"Transaction amount: INR 250000"
h_original = hashlib.sha256(original).hexdigest()
tampered = b"Transaction amount: INR 950000"
h_tampered = hashlib.sha256(tampered).hexdigest()
print("Integrity check:", "PASSED" if h_original == h_tampered else "FAILED -
modification detected")

```

Strategy C – Trusted Cryptographic Security

All prior mechanisms are combined with RSA digital signatures (sign-then-encrypt) for non-repudiation and an X.509 certificate issued for the sender's public key, providing a trusted, verifiable identity binding in addition to RSA + Diffie–Hellman + hashing.

```

"""
Strategy C - Trusted Cryptographic Security
RSA + Diffie-Hellman + Cryptographic Hash + Digital Signature + X.509
"""
import datetime, hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, dh, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography import x509
from cryptography.x509.oid import NameOID

# 1. DH key exchange + SHA-256-derived session key (as in Strategy B)
parameters = dh.generate_parameters(generator=2, key_size=2048)
sender_priv = parameters.generate_private_key()
receiver_priv = parameters.generate_private_key()
shared_secret = sender_priv.exchange(receiver_priv.public_key())
session_key = HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
    info=b"txn-session").derive(shared_secret)

# 2. RSA key pair, digital signature over the transaction hash
rsa_priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
rsa_pub = rsa_priv.public_key()
txn = b"Fund transfer: A/C 100234 -> A/C 100987, Amount INR 250000"
txn_hash = hashlib.sha256(txn).digest()
signature = rsa_priv.sign(txn_hash, padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256())

# 3. Verification (integrity + authentication + non-repudiation)
try:
    rsa_pub.verify(signature, txn_hash, padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256())
    print("Signature verification: VALID")
except Exception:
    print("Signature verification: INVALID")

# 4. X.509 certificate binding sender identity to the RSA public key
subject = issuer = x509.Name([

```

```

x509.NameAttribute(NameOID.COUNTRY_NAME, "IN"),
x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Financial Services Org"),
x509.NameAttribute(NameOID.COMMON_NAME, "txn-analyst.financeorg.local"),
])
cert = (x509.CertificateBuilder()
        .subject_name(subject).issuer_name(issuer).public_key(rsa_pub)
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.datetime.now(datetime.timezone.utc))
        .not_valid_after(datetime.datetime.now(datetime.timezone.utc) +
                        datetime.timedelta(days=365))
        .sign(rsa_priv, hashes.SHA256()))
print("Certificate subject:", cert.subject.rfc4514_string())

# 5. Attack scenario - impersonation
impostor_priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
forged_signature = impostor_priv.sign(txn_hash, padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256())
try:
    rsa_pub.verify(forged_signature, txn_hash, padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256())
    print("Impostor signature accepted (unexpected)")
except Exception:
    print("Impersonation attempt rejected - signature does not match certified public
key")

```

More than one solution was considered: Strategy B was evaluated as an intermediate approach that fixes RSA's key-distribution weakness through Diffie–Hellman, while Strategy A was retained as a baseline representing minimal, conventional public-key protection.

4. Use of Modern Tools

- Python (cryptography library) – to implement RSA encryption/decryption, Diffie–Hellman key exchange, SHA-256 hashing, RSA digital signatures and X.509 certificate generation/verification.
- OpenSSL – to independently generate and inspect RSA keys and X.509 certificates for verification of the Strategy C implementation.
- Wireshark – to capture and inspect simulated transaction traffic for analysis of confidentiality (encrypted payload) versus plaintext exposure.

Command logs, key/certificate screenshots and captured traffic evidence are to be attached in the appendix of the submitted report.

5. Results and Validation

Each strategy was evaluated against ten criteria — confidentiality, integrity, authentication, non-repudiation, key-management complexity, computational efficiency, scalability, security strength, resistance to attacks and reliability — on a 1–5 relative rating scale, informed by the properties of the underlying mechanisms and the execution results above.

Criterion	Strategy A	Strategy B	Strategy C
Confidentiality	3	4	5
Integrity	1	4	5
Authentication	2	3	5
Non-repudiation	1	1	5
Key-management complexity (lower = simpler)	2	3	4
Computational efficiency	4	3	2
Scalability	2	3	5
Security strength	2	4	5
Resistance to attacks	2	4	5
Reliability	3	4	5

Validation: Strategy A provides confidentiality but no integrity, authentication or non-repudiation guarantee, and depends on prior secure distribution of the receiver's public key. Strategy B adds secure, forward-secret key establishment and integrity checking via hashing but still lacks non-repudiation. Strategy C satisfies all ten evaluated criteria by combining RSA, Diffie–Hellman, hashing, digital signatures and X.509 authentication, at the cost of higher computational overhead and key-management complexity.

6. Analysis and Engineering Decision

Comparing the alternatives:

- Strategy A is computationally efficient and simple to manage but offers only confidentiality — acceptable only for low-risk, non-critical internal communication.

- Strategy B substantially improves key establishment (forward secrecy via Diffie–Hellman) and adds integrity checking, but still lacks a verifiable, non-repudiable link between a signature and an identity.
- Strategy C provides the most complete protection across every evaluated criterion by combining RSA, Diffie–Hellman, hashing, digital signatures and X.509 authentication, at the cost of the highest computational and administrative overhead.

Trade-off identified: security strength and computational efficiency move in opposite directions across the three strategies — the highest-security option (Strategy C) is also the most computationally and administratively demanding (certificate lifecycle management, signature generation/verification overhead), while the simplest option (Strategy A) is the most efficient but leaves transactions unauthenticated and repudiable.

Engineering decision: Strategy C is recommended for the financial organization, since it is the only strategy that satisfies all identified requirements — confidentiality, integrity, authentication and non-repudiation — for an organization handling sensitive customer and financial transactions. The added computational and administrative cost is justified by the significantly reduced risk of unauthorized access, transaction tampering and impersonation, and can be mitigated through phased rollout and centralized certificate/key management.

7. Broader Considerations

- Decent Work and Economic Growth (SDG 8): trusted, authenticated digital transactions support reliable financial services and safeguard the economic activity and livelihoods that depend on them.
- Responsible Consumption and Production (SDG 12): standards-based, reusable cryptographic infrastructure (shared PKI, certificate authority) avoids duplicated, ad hoc security implementations across branches and systems.
- Ethics and privacy: private-key and certificate management must be governed by clear, lawful policy to protect customer data and avoid misuse of authentication authority.
- Economics: higher-security strategies incur greater PKI licensing, certificate-authority and administrative cost, which must be weighed against the potential cost of a security breach.

- Professional responsibility: as Cryptography and Network Security Analyst, recommending the strategy that best protects customer and transaction data reflects due professional diligence.

8. Conclusion

Three cryptographic security strategies of increasing sophistication were designed, implemented and evaluated: Strategy A (RSA encryption with a public/private key pair), Strategy B (RSA with Diffie–Hellman key exchange and hashing) and Strategy C (RSA, Diffie–Hellman, hashing, digital signatures and X.509 authentication). Evaluation against confidentiality, integrity, authentication, non-repudiation, key-management complexity, computational efficiency, scalability, security strength, resistance to attacks and reliability, backed by working Python implementations of all three strategies and simulated attack scenarios, showed that Strategy C provides the most complete protection and best satisfies the financial organization's requirements, despite higher computational and administrative overhead.

Final Verdict: Best Strategy = Strategy C – Trusted Cryptographic Security

Strategy C scored highest on every security criterion evaluated and was the only strategy whose implementation both detected message modification and rejected an impersonation attempt using a forged signature, while also binding the sender's identity to its public key via an X.509 certificate. It is recommended for enterprise deployment, with Strategy B as an acceptable lower-overhead fallback and Strategy A suitable only for low-risk, non-critical internal use.

Limitations of this study include the use of simulated rather than live production transaction data for performance evaluation, and reliance on representative rather than measured latency figures at enterprise scale; future work could validate the architecture using a live testbed and formal penetration testing.

9. Student Reflection

1. Which security mechanism required the highest level of evaluation? The X.509 certificate infrastructure and RSA digital-signature verification in Strategy C required the deepest evaluation, since it involved certificate issuance, identity binding and signature-verification logic — substantially more analysis than the single RSA encrypt/decrypt operation in Strategy A.

2. What major trade-off was identified? The trade-off between security strength and computational/administrative efficiency: Strategy C offers the strongest protection across every criterion evaluated but is the most demanding to compute and administer, while Strategy A is the most efficient but leaves transactions unauthenticated and repudiable.

3. Which security mechanism provided the greatest improvement in protection? Adding RSA digital signatures and X.509 authentication on top of RSA + Diffie–Hellman + hashing produced the largest single improvement, since it addressed authentication and non-repudiation simultaneously — the criteria where Strategies A and B scored weakest.

4. What did you learn from this assignment beyond what was directly taught in the classroom? This assignment showed how RSA, Diffie–Hellman, hashing, digital signatures and X.509 authentication must be combined into a single coherent security architecture, and how real deployment decisions require weighing security gains against computational cost and key-management overhead rather than simply choosing the strongest available mechanism.

5. If you were given additional time/resources, what would you improve and why? With more time, the evaluation would be extended from representative ratings to a live testbed with measured encryption/signature latency and throughput under concurrent load, and would include formal penetration testing to validate the architecture's resilience against real attack traffic.

10. References

- Stallings, W. Cryptography and Network Security: Principles and Practice.
- RFC 5280 – Internet X.509 Public Key Infrastructure Certificate and CRL Profile.
- RFC 2631 / RFC 3526 – Diffie–Hellman Key Agreement Method / MODP Groups.

- PKCS #1 – RSA Cryptography Standard.
- NIST FIPS 180-4 – Secure Hash Standard (SHS).
- Python cryptography library and OpenSSL official documentation.
- Course lecture notes, CSA51 – Cryptography and Network Security.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level
Problem formulation	CO2, CO3	PO1, PO2	L4/L5
Application of knowledge	CO2, CO3	PO1, PO2	L4/L5
Solution / Design / Implementation	CO2, CO3	PO1, PO3, PO5	L4/L5/L6
Modern tool usage	CO2, CO3	PO5	L3/L4
Validation	CO2, CO3	PO4, PO5	L4/L5
Analysis & justification	CO2, CO3	PO2, PO4	L5
Broader considerations	CO3	PO6, PO7, PO8	L4/L5
Documentation & reflection	-	PO10, PO12	L3/L4

PO reference (NBA graduate attributes): PO1 – Engineering knowledge; PO2 – Problem analysis; PO3 – Design/development of solutions; PO4 – Conduct investigations of complex problems; PO5 – Modern tool usage; PO6 – The engineer and society; PO7 – Environment and sustainability; PO8 – Ethics; PO9 – Individual and team work; PO10 – Communication; PO11 – Project management and finance; PO12 – Life-long learning.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering:

TITLE: Evaluation and Design of an Integrated Public Key and Authentication Security System

CONTRIBUTION:

S. No	REG NO	NAME	CONTRIBUTION
1.	192311436	B. Veera Harsha Vardhan Reddy	Led the problem formulation and requirement analysis; designed the overall system architecture and public key infrastructure; implemented public key encryption/decryption and key generation modules; contributed to testing and result analysis.
2.	192511154	M Nishanth	Designed and implemented the password-based authentication and secure hashing modules; developed digital signature generation and verification; contributed to authentication security analysis and execution validation.
3.	192411210	Ch. Mahi Vardhan Reddy	Implemented certificate-based authentication and public key verification mechanisms; integrated the different security modules into the proposed system; performed security testing and prepared execution screenshots/results.
4.	192524269	Thummuru Dileep	Conducted comparative evaluation and trade-off analysis of the proposed security mechanisms; prepared the final strategy/recommendations, broader considerations and conclusion; compiled references, formatted the report, and prepared the CO–PO assessment mapping.